

CLAUDE CODE MASTERCLASS

42 techniques pour diviser par 5 vos tokens

Le guide que personne ne partage

Architecture prompt, routage de modèles, skills on-demand, MCP
servers, RTK, agents parallèles.

Chaque technique chiffrée. Avant/après. Formules. Erreurs
courantes.

Applicable à Claude Code, Claude Projects, GPT, Gemini.

42

!

Le problème

Tu payes tes tokens au poids. Chaque mot dans le system prompt, chaque sortie de commande, chaque fichier lu entre dans la facture. La plupart des utilisateurs de Claude Code consomment **3 à 5 fois plus que nécessaire**.

5x

C'est le ratio moyen entre un setup optimisé et un setup par défaut. Sur un projet actif, ça représente des centaines d'euros par mois.

La formule

```
tokens/session =  
  CLAUDE.md // chargé à chaque message  
+ skills chargées // on-demand ou toujours ?  
+ références lues // 168 KB ou 5 KB ?  
+ mémoire indexée // index ou contenu entier ?  
+ sorties commandes // brutes ou filtrées ?  
+ réponses outils // curl verbose ou MCP structuré ?  
+ itérations gaspillées // plan mode ou trial & error ?
```

Chaque ligne de cette formule est un levier. Ce document attaque les 7.

Architecture en couches

CLAUDE.md	Chargé à CHAQUE message – chaque mot compte
Skills (on-demand)	Chargés uniquement au match – 0 si non déclenchées
Références	Chargés par les skills – jamais en bloc
Mémoire (MEMORY.md)	Index chargé, fichiers lus à la demande
MCP + RTK	Réduisent les sorties AVANT qu'elles entrent dans le contexte

Principe fondamental : tout ce qui peut être chargé à la demande ne doit pas être chargé par défaut. Chaque couche filtre ce qui arrive à la suivante.

FAMILLE A

Architecture du prompt system

7 techniques pour que le fichier CLAUDE.md et la mémoire ne bouffent pas la moitié de votre budget tokens avant même de commencer à travailler.

A Architecture du prompt system

#1 CLAUDE.md en tables, pas en prose

AVANT — PROSE (~400 LIGNES)

Pour récupérer les données de votre API, utilisez un serveur MCP dédié. Ne faites jamais de curl avec des headers d'auth manuels...

APRES — TABLE (~113 LIGNES)

| Données API | mon-mcp (N outils) | Pas de curl |

-70%

#2 Rulebooks thématiques au lieu d'un fichier géant

7 fichiers de 48-72 lignes, chargés selon la tâche. Écriture de contenu ? Charger rules-content + rules-editorial. Debug serveur ? Charger rules-technique. Jamais les 7 en même temps.

-50%

#3 MEMORY.md en index, pas en contenu

AVANT — TOUT INLINE

3 000+ lignes de règles, historique, décisions, contexte → ~50 000 tokens chargés à chaque session

APRES — INDEX + POINTEURS

94 lignes de liens vers 7 rulebooks + fichiers mémoire. Claude lit un fichier seulement quand il en a besoin. ~1 500 tokens.

-97%

#4 Références lourdes dans un dossier séparé

Votre documentation technique dans `.claude/references/`. Jamais chargée en bloc. Chaque skill déclare exactement ce dont elle a besoin.

-90%

#5 Sous-dossiers thématiques dans les références

Pas un dossier plat de 14 fichiers. Des sous-dossiers : `seo/`, `content/`, `devops/`. L'IA charge 400 lignes au lieu de 3 900.

-84%

#6 Interdictions explicites, pas d'explications

AVANT — NARRATIF (~400 TOKENS)

"Vous devriez faire attention à ne pas publier directement car une fois, un article a été publié sans review et le contenu était de mauvaise qualité..."

APRÈS — RÈGLE (~50 TOKENS)

JAMAIS publier un article. Le pipeline s'arrête au brouillon.

-87%

#7 Checklists au lieu de prose procédurale

82 lignes de checklist remplacent 350 lignes de "Quand vous êtes sur le point de publier, commencez par..."

-71%

B Routage de modèles

Un seul modèle pour tout = gaspillage. Chaque tâche a un coût optimal.

Opus — le chirurgien

Design stratégique, rédaction complexe, audits multi-domaines

Sonnet — le couteau suisse

Admin, debug, contenu standard, API calls

Haiku — le rapide

Audits read-only, changelogs, tâches mécaniques

#8 Haiku pour les tâches read-only

Un agent dédié fait des audits responsive et accessibilité. Il lit, il compare, il note. Zéro code généré. Haiku suffit.

-10 à 50% vs Sonnet

#9 Sonnet pour le standard

Vos agents standard tournent en Sonnet : admin (serveur, BDD), content (rédaction + API), debugger (diagnostic). Le sweet spot coût/capacité.

#10 Opus uniquement quand justifié

1 seul agent en Opus pour le travail stratégique : design, architecture, décisions complexes. Le genre de tâche où le raisonnement complexe fait une vraie différence.

#11 Modèle déclaré dans le YAML de chaque agent

SANS DÉCLARATION

"Lance un agent pour faire un audit design. Utilise Haiku parce que c'est du read-only..." — ~200 tokens de négociation à chaque lancement.

AVEC YAML

model: haiku dans le fichier agent. Zéro négociation. Le bon modèle à chaque fois.

IMPACT RÉEL

Sur un projet avec 20 lancements d'agents par semaine : **~4 000 tokens économisés** juste en évitant la négociation de modèle.

C Système de skills

#12 13 micro-skills au lieu d'1 méga-prompt

MEGA-PROMPT (~100 KB)

Tout dans CLAUDE.md : règles SEO, pipeline d'écriture, audit, backup, media, changelog... Chargé à CHAQUE message.

13 MICRO-SKILLS (4-15 KB CHACUNE)

/audit , /deploy , /backup , /changelog ...
Chargées uniquement quand déclenchées. 12 sur 13
= 0 token.

Chargé 1/13e

#13 Chargement on-demand via trigger evaluation

Seul le metadata de chaque skill (nom + description) est indexé dans le system prompt. Le contenu complet (4-15 KB) se charge uniquement au match. Résultat : ~200 KB évités par requête qui ne déclenche aucune skill.

#14 Framework de test des triggers

20 queries de test dans un JSON : 12 doivent déclencher une skill, 8 ne doivent rien déclencher. Ça évite les faux positifs (= chargement inutile de skill).

Exemple de test

"lance un audit complet" → **déclenche /audit**

"déploie le frontend en prod" → **ne déclenche rien** (zéro skill chargée)

#15 Chaque skill déclare ses références exactes

/audit charge scoring.md + benchmarks.md . Pas les 14 fichiers de références. L'isolation évite 50-100 KB de contexte inutile par invocation.

#16 Références par skill dans des sous-dossiers

Chaque skill a son propre dossier références/ avec ses fichiers spécifiques : mots-clés suivis, scoring matrices, benchmarks. Pas de mélange.

#17 Données persistantes entre sessions

Les benchmarks (seuils de performance, KPIs cibles, mots-clés suivis) sont dans des fichiers JSON/MD. Pas besoin de "rappelle-moi les seuils" à chaque session. 500-800 tokens économisés par session.

D RTK — Rust Token Killer

Un outil CLI qui compresse les sorties de commandes **avant** qu'elles entrent dans le contexte de Claude. Tu préfixes tes commandes avec `rtk`. Le reste est automatique.

#18 Tests : seuls les échecs

SANS RTK — VITEST RUN

500 lignes : 247 tests passés (détaillés), 2 échecs, durée par fichier, couverture...

AVEC RTK VITEST RUN

3 lignes : les 2 échecs, fichier + ligne + message. C'est tout.

-90 à 99%

#19 Build : erreurs groupées par fichier

`rtk tsc` : TypeScript groupée par fichier/code. `rtk next build` : métriques routes uniquement. Plus de bruit de progression.

-70 à 87%

#20 Git : format compact

`rtk git diff` supprime le contexte inutile. `rtk git log` = compact. `rtk git status` = compact. Même `rtk git add` et `rtk git commit` sont compressés.

-59 à 80%

#21 GitHub : metadata seulement

`rtk gh pr view` supprime le diff complet, garde le titre, statut, checks. `rtk gh run list` = compact.

-26 à 87%

#22 Package managers : arbre compact

`rtk pnpm install` supprime les lignes de progression. `rtk pnpm list` = arbre compact. `rtk prisma` = sans ASCII art.

-70 à 90%

#23 Monitoring des économies

`rtk gain` affiche les statistiques d'économie. `rtk discover` analyse les sessions Claude Code pour détecter les commandes qui auraient dû utiliser RTK. Méta-optimisation.

IMPACT CUMULÉ RTK

5 commandes par session × 500 tokens moyens = 2 500 tokens bruts. Avec RTK : ~625 tokens.

Économie : ~1 875 tokens par session.

E Accès aux données externes

Le problème de base : Claude a besoin de données externes (API, BDD, services). La solution naïve (WebSearch + curl brut) explose le budget tokens. Deux approches modernes existent. Aucune n'est universellement meilleure.

#24 Le vrai ennemi : curl brut et WebSearch

Avant de choisir entre MCP et CLI, rappelons ce qu'il faut éliminer. Un `curl` avec headers d'auth manuels + parsing de réponse JSON brute = 300-500 tokens. Un `WebSearch` suivi d'un `WebFetch` pour lire une page de docs = 500-1 000 tokens. C'est ça qu'on remplace.

Les deux approches

APPROCHE MCP SERVER

Principe : un serveur local expose les outils via le protocole MCP. Claude les appelle comme des fonctions natives.

Exemples : context7 (docs framework), google-search-console, votre API métier en MCP custom.

Sortie : JSON structuré, typé. Claude reçoit exactement les champs utiles.

APPROCHE CLI WRAPPÉ

Principe : un outil CLI existant (gh, gcloud, aws, vercel, npm) appelé via Bash, avec un filtre de sortie (RTK, jq, grep).

Exemples : `rtk gh pr view`, `aws s3 ls | head`, `vercel ls --json | jq`.

Sortie : texte filtré. Aussi compact qu'un MCP si le wrapper est bon.

#25 Quand choisir MCP

L'API n'a pas de CLI officiel

Pas de CLI Google Search Console. Pas de CLI pour votre API métier interne. MCP est le seul moyen d'éviter du curl brut.

L'auth est complexe (OAuth, tokens rotatifs)

MCP gère l'auth en interne via env vars. Zéro token d'`Authorization: Bearer ...` dans le contexte. Zéro refresh token à gérer.

Le schéma des outils guide Claude

MCP expose le schéma JSON des paramètres. Claude sait quoi passer sans qu'on lui explique dans le prompt. Moins de négociation = moins de tokens.

#26 Quand choisir CLI wrappé

Un CLI officiel existe et il est bon

`gh` (GitHub), `gcloud`, `aws`, `vercel`, `npm`, `docker`. Maintenus par l'éditeur. Toujours à jour. Pas de serveur à maintenir.

#27 L'arbre de décision

L'outil a un CLI officiel maintenu ?	
OUI → CLI wrappé (+ RTK ou jq)	gh, gcloud, aws, vercel, docker, npm
NON → MCP server existe sur le marché ?	
OUI → Installer le MCP	context7, GSC, GA4, Stripe, Supabase...
NON → Écrire un MCP custom ou un wrapper Bash	API interne, BDD custom, service maison

#28 Comparaison tokens : même tâche, 3 méthodes

Exemple : récupérer les 5 dernières pull requests d'un repo GitHub.

MÉTHODE	COMMANDE / APPEL	TOKENS EN SORTIE
curl brut	<code>curl -H "Authorization: token ..." api.github.com/repos/.../pulls</code>	~2 000 (JSON complet, 50+ champs par PR)
MCP GitHub	<code>mcp_github_list_pull_requests</code>	~400 (champs filtrés par le serveur)
CLI wrappé	<code>rtk gh pr list --limit 5</code>	~150 (RTK compresse titre+statut+auteur)

LE POINT CLÉ

Ce qui compte pour les tokens, c'est la **taille de la sortie qui entre dans le contexte**. Un CLI bien wrappé peut battre un MCP mal configuré. Un MCP bien fait peut battre un CLI sans filtre. L'outil n'est pas la réponse. Le filtrage l'est.

#29 Le cas context7 : MCP imbattable

Pour les docs de frameworks (Next.js, React, Tailwind, Django...), context7 est un cas où le MCP est clairement supérieur. Un `WebSearch "Next.js App Router cache"` renvoie 3 pages de résultats obsolètes. Context7 renvoie la doc à jour, ciblée, en 200 tokens.

#30 Le cas GitHub : CLI wrappé imbattable

`gh` est le CLI officiel, maintenu par GitHub, toujours à jour. Avec RTK : `rtk gh pr view 123` = -87%. Installer un MCP GitHub en plus serait redondant et moins compact.

#31 Credentials : même logique dans les deux cas

MCP : env vars dans `.mcp.json`. CLI : env vars ou fichier `~/.config`. Dans les deux cas, zéro token de credentials dans le contexte. C'est le curl brut avec `-H "Authorization: ..."` qui est le vrai problème.

Avant/après : accès données analytiques

NAÏF — ~6 000 TOKENS

WebSearch "GSC API" + lire la doc + construire le curl + parser la réponse JSON + recommencer pour GA4 + recommencer pour Bing

MCP OU CLI WRAPPÉ — ~1 500 TOKENS

3 appels structurés (MCP ou CLI). Sortie filtrée. Pas de doc à relire.

-75% quel que soit le choix MCP/CLI

F Workflow et process

#32 Team templates pré-écrits

4 templates d'équipes réutilisables dans `teams.md` : "Review", "Publication + Review", "Debug Multi-Hypothèses", "Refonte Multi-Pages". Plus besoin de décrire qui fait quoi à chaque fois.

-75% vs décrire chaque équipe

#33 Commande audit-complet : 4 agents parallèles

Une seule commande `/audit-full` lance 4 agents en parallèle (un par domaine). Chaque agent a son contexte minimal. Pas de méga-session qui charge tout.

#34 Plan mode avant exécution

Planifier avant de coder évite les itérations gaspillées. Un plan de 500 tokens peut économiser 5 000 tokens d'essai-erreur.

#35 Sessions indexées, pas transcrites

Vos sessions résumées en 1-2 lignes chacune dans `sessions.md`. Pas de transcripts complets.

-98%

#36 Fichier de reprise de session

Un fichier de reprise : contexte structuré pour reprendre le travail. Remplace `git log --oneline -50` + `git status` + relecture de fichiers.

-88%

#37 Passation Git entre machines

La mémoire est dans le repo Git. Changer de machine = `git pull`. Zéro token dépensé pour "rappelle-moi où on en était".

#38 Single source of truth pour les patterns

Un document unique sert de référence pour les patterns récurrents. Pas de duplication dans le prompt. Claude le fetch quand il en a besoin.

#39 NotebookLM comme RAG externe

Vos sources indexées dans un RAG externe (NotebookLM, Pinecone, etc.). Questions d'architecture ou conventions : interroger le RAG au lieu de lire 50 fichiers dans le contexte.

-80% sur les questions d'architecture

Credentials et sécurité

#40 Credentials dans un fichier .gitignore

`config/credentials.json` stocke tout : clés API, credentials BDD, tokens OAuth, secrets divers. Chargé au runtime par les MCP ou CLI. Jamais dans le prompt.

#41 MCP/CLI gère l'auth en interne

Les env vars sont dans `.mcp.json` ou `~/.config`. Claude n'a jamais besoin de construire un header `Authorization: Basic ...`. Zéro token d'auth dans le contexte.

#42 Scripts temporaires auto-supprimés

Règle d'hygiène : tout script temporaire est supprimé après exécution. Pas d'artefact, pas de discussion "faut-il supprimer le script ?".

! Les 7 erreurs courantes

Erreur #1 : Tout mettre dans CLAUDE.md

Le fichier est chargé à chaque message. 500 lignes = ~7 000 tokens brûlés à chaque échange. Garder < 130 lignes. Le reste en skills et références.

Erreur #2 : Un seul modèle pour tout

Opus pour un changelog ? Sonnet pour un audit read-only ? Chaque tâche a son modèle optimal. Définir dans le YAML de l'agent.

Erreur #3 : Pas de skills, tout dans le prompt

Sans skills, chaque instruction est chargée à chaque message. Avec skills, 12 workflows sur 13 coûtent 0 token quand ils ne sont pas actifs.

Erreur #4 : Laisser les commandes cracher du brut

`git log` sans filtre = 2 000 tokens. Avec RTK = 400. Multiplier par 10 commandes/session.

Erreur #5 : curl brut au lieu de MCP ou CLI wrappé

Un curl avec auth headers + JSON body + parsing = 300-500 tokens. Un appel MCP ou CLI wrappé = 50-150 tokens. Même résultat.

Erreur #6 : Mémoire en vrac

Un fichier mémoire de 3 000 lignes chargé en entier à chaque session. Préférer un index (94 lignes) + des fichiers spécifiques chargés à la demande.

Erreur #7 : Pas de plan mode

Coder directement = itérations d'essai-erreur. Un plan de 500 tokens peut économiser 5 000 tokens de corrections. Planifier d'abord, toujours.

\$

Coût réel : 3 scénarios

Scénario 1 : Audit mensuel

ÉTAPE	SANS OPTIMISATION	AVEC OPTIMISATION
System prompt	~7 000 tokens (CLAUDE.md massif)	~1 500 tokens (compact + index)
Skills chargées	0 (tout dans le prompt)	~2 000 tokens (1 skill ciblée)
Données analytiques	~3 000 tokens (WebSearch x6)	~800 tokens (3 MCP calls)
Commandes git/build	~2 500 tokens (brut)	~625 tokens (RTK)
Itérations	~3 000 tokens (pas de plan)	~500 tokens (plan mode)
TOTAL	~15 500 tokens	~5 425 tokens



Scénario 2 : Création de contenu via API

ÉTAPE	SANS OPTIMISATION	AVEC OPTIMISATION
System prompt	~7 000 tokens	~1 500 tokens
Recherche packages	~2 000 tokens (WebFetch x5)	~400 tokens (MCP registry)
Création contenu	~1 500 tokens (curl + auth + JSON)	~300 tokens (MCP API)
Référence patterns	~2 000 tokens (re-expliquer)	~200 tokens (fetch référence unique)
Règles métier	~3 000 tokens (tout inline)	~800 tokens (2 rulebooks)
TOTAL	~15 500 tokens	~3 200 tokens



Scénario 3 : Debug serveur

ÉTAPE	SANS OPTIMISATION	AVEC OPTIMISATION
System prompt	~7 000 tokens	~1 500 tokens
Logs serveur	~3 000 tokens (brut)	~500 tokens (rtk log)
Tests hypothèses	~4 000 tokens (séquentiel)	~1 500 tokens (agents parallèles)
Git operations	~1 500 tokens (brut)	~400 tokens (rtk git)
TOTAL	~15 500 tokens	~3 900 tokens



~34K

tokens économisés par session en moyenne. Sur 20 sessions par semaine, ça représente ~680 000 tokens/semaine qui ne partent pas en fumée.

✓ Checklist d'implémentation

Par où commencer ? Dans l'ordre d'impact. Les 5 premières prennent 30 minutes et couvrent 80% des gains.

IMMÉDIAT (30 MIN, 80% DES GAINS)

- ☐ Compresser CLAUDE.md en tables (< 130 lignes)
- ☐ Installer RTK et préfixer toutes les commandes
- ☐ Extraire les règles métier en skills séparées
- ☐ Créer un MEMORY.md index (pointeurs, pas contenu)
- ☐ Installer les MCP servers pour vos API les plus utilisées

SEMAINE 1 (OPTIMISATION FINE)

- ☐ Configurer des agents avec modèles assignés
- ☐ Créer des team templates pour les workflows récurrents
- ☐ Séparer les références lourdes dans des sous-dossiers
- ☐ Activer le plan mode par défaut pour les tâches complexes
- ☐ Tester les triggers de skills (JSON eval)

SEMAINE 2 (AUTOMATISATION)

- ☐ Créer un fichier de reprise de session (A-LIRE-EN-PREMIER)
- ☐ Indexer les sessions passées (summaries, pas transcripts)
- ☐ Configurer la passation Git entre machines
- ☐ Externaliser le contexte architecture vers un RAG
- ☐ Écrire les interdictions explicites (rules-never.md)

MONITORING CONTINU

- ☐ Lancer `rtk gain` chaque semaine
- ☐ Lancer `rtk discover` chaque mois
- ☐ Vérifier la taille du CLAUDE.md après chaque ajout
- ☐ Supprimer les skills inutilisées depuis 30 jours
- ☐ Mettre à jour les rulebooks quand les règles changent

42 Récap complet

FAMILLE	#	TECHNIQUE	ÉCONOMIE
A. Prompt	1	CLAUDE.md en tables	70%
	2	Rulebooks thématiques	50%
	3	MEMORY.md en index	97%
	4	Références séparées	90%
	5	Sous-dossiers thématiques	84%
	6	Interdictions explicites	87%
	7	Checklists vs prose	71%
B. Modèles	8	Haiku read-only	10-50%
	9	Sonnet standard	Baseline
	10	Opus justifié	Contrôle
	11	Modèle dans le YAML	~200 tok/agent
C. Skills	12	13 micro-skills	Chargé 1/13e
	13	Chargement on-demand	~200 KB évités
	14	Test triggers JSON	Zero faux positif
	15	Références par skill	50-100 KB
	16	Sous-dossiers par skill	Isolation
	17	Données persistantes	500-800/session
D. RTK	18	Tests : échecs seuls	90-99%
	19	Build : erreurs groupées	70-87%
	20	Git : format compact	59-80%
	21	GitHub : metadata	26-87%
	22	Package managers	70-90%
	23	Monitoring RTK	Meta
E. Données	24	Éliminer curl brut + WebSearch	60-80%
	25	MCP quand pas de CLI officiel	60-70%
	26	CLI wrappé quand CLI existe	50-87%
	27	Arbre de décision MCP vs CLI	Aide au choix
	28	Comparaison 3 méthodes	Benchmark

LE PRINCIPE

Tout ce qui peut être charge à la demande ne doit pas être charge par défaut.

7 familles. 42 techniques. 3 scénarios chiffrés.

Une checklist d'implémentation en 4 phases.

Résultat : -65 à -79% de tokens par session.

Retrouve-moi

wpformation.com

Blog WordPress et IA

Fabrice Ducarme

LinkedIn

42